

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3
Специальность ПО-13

Выполнил:
студент 3 курса
группы ПО-13
А.В. Король
Проверил:
А.Д.Кулик
01.04.2026

Брест 2026

Цель работы: приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python

Задание 1.

9) Проект «Бургер-закусочная». Реализовать возможность формирования заказа из определенных позиций (тип бургера (веганский, куриный и т.д.)), напиток (холодный – пепси, кока-кола и т.д.; горячий – кофе, чай и т.д.), тип упаковки – с собой, на месте. Должна формироваться итоговая стоимость заказа.

Выполнение:

Код программы:

```
class Burger:
    def __init__(self, name, price):
        self.name = name
        self.price = price

class Drink:
    def __init__(self, name, price):
        self.name = name
        self.price = price

class Order:
    def __init__(self):
        self.items = []

    def add_item(self, item):
        self.items.append(item)

    def total(self):
        return sum(item.price for item in self.items)

    def show(self):
        for item in self.items:
            print(f"{item.name} - {item.price}")

        print(f"Итого: {self.total()}")

class Factory:
    @staticmethod
    def create_burger(choice):
        burgers = {
            "1": Burger("Chicken Burger", 300),
            "2": Burger("Vegan Burger", 250),
            "3": Burger("Cheese Burger", 350),
        }
```

```

        return burgers.get(choice)

    @staticmethod
    def create_drink(choice):
        drinks = {
            "1": Drink("Cola", 100),
            "2": Drink("Tea", 80),
            "3": Drink("Coffee", 120),
        }

        return drinks.get(choice)

order = Order()

print("Бургеры:")
print("1 - Chicken Burger")
print("2 - Vegan Burger")
print("3 - Cheese Burger")

burger_choice = input("Выберите бургер: ")

burger = Factory.create_burger(burger_choice)

if burger:
    order.add_item(burger)

print("\nНапитки:")
print("1 - Cola")
print("2 - Tea")
print("3 - Coffee")

drink_choice = input("Выберите напиток: ")

drink = Factory.create_drink(drink_choice)

if drink:
    order.add_item(drink)

print("\nВаш заказ:")
order.show()

```

Рисунки с результатами работы программы

```
PS C:\Users\user\Docume
Бургеры:
1 - Chicken Burger
2 - Vegan Burger
3 - Cheese Burger
Выберите бургер: 2

Напитки:
1 - Cola
2 - Tea
3 - Coffee
Выберите напиток: 2

Ваш заказ:
Vegan Burger - 250
Tea - 80
Итого: 330
```

Задание 2.

9) Проект «Часы». В проекте должен быть реализован класс, который дает возможность пользоваться часами со стрелками так же, как и цифровыми часами. В классе «Часы со стрелками» хранятся повороты стрелок.

Выполнение:

Код программы:

```
class AnalogClock:
    def __init__(self, hour_angle):
        self.hour_angle = hour_angle

    def get_angle(self):
        return self.hour_angle

class ClockAdapter:
    def __init__(self, analog_clock):
        self.analog_clock = analog_clock

    def get_time(self):
        hours = int(self.analog_clock.get_angle() / 30)

        if hours == 0:
            hours = 12

        return f"{hours}:00"

angle = int(input("Введите угол часовой стрелки: "))

clock = AnalogClock(angle)

adapter = ClockAdapter(clock)

print("Время:", adapter.get_time())
```

Рисунки с результатами работы программы

```
PS C:\Users\user\Documents\GitHub\s
Введите угол часовой стрелки: 160
Время: 5:00
```

Задание 3.

9) Шифрование текстового файла. Реализовать класс-шифровщик текстового файла с поддержкой различных алгоритмов шифрования. Возможные варианты шифрования: удаление всех гласных букв из текста, изменение букв текста на буквы, получаемые фиксированным сдвигом из алфавита (например, шифром буквы а будет являться буква д для сдвига 4 и т.д.), применение операции исключающее или с заданным ключом.

Выполнение:

Код программы:

```
class EncryptStrategy:
    def encrypt(self, text):
        raise NotImplementedError

class RemoveVowels(EncryptStrategy):
    def encrypt(self, text):
        vowels = "aeiouAEIOUaeёиоуыэяАЕЁИОУЫЭЯ"

        return "".join(char for char in text if char not in vowels)

class CaesarCipher(EncryptStrategy):
    def __init__(self, shift):
        self.shift = shift

    def encrypt(self, text):
        result = ""

        for char in text:
            result += chr(ord(char) + self.shift)

        return result

class Encryptor:
    def __init__(self, strategy):
        self.strategy = strategy

    def encrypt(self, text):
        return self.strategy.encrypt(text)

file_name = input("Введите имя txt файла: ")
```

```

with open(file_name, "r", encoding="utf-8") as file:
    text = file.read()

print("1 - Удалить гласные")
print("2 - Шифр Цезаря")

choice = input("Выберите метод: ")

if choice == "1":
    encryptor = Encryptor(RemoveVowels())

    result = encryptor.encrypt(text)

    with open("without_vowels.txt", "w", encoding="utf-8") as file:
        file.write(result)

    print("Файл сохранен: without_vowels.txt")

elif choice == "2":
    shift = int(input("Введите сдвиг: "))

    encryptor = Encryptor(CaesarCipher(shift))

    result = encryptor.encrypt(text)

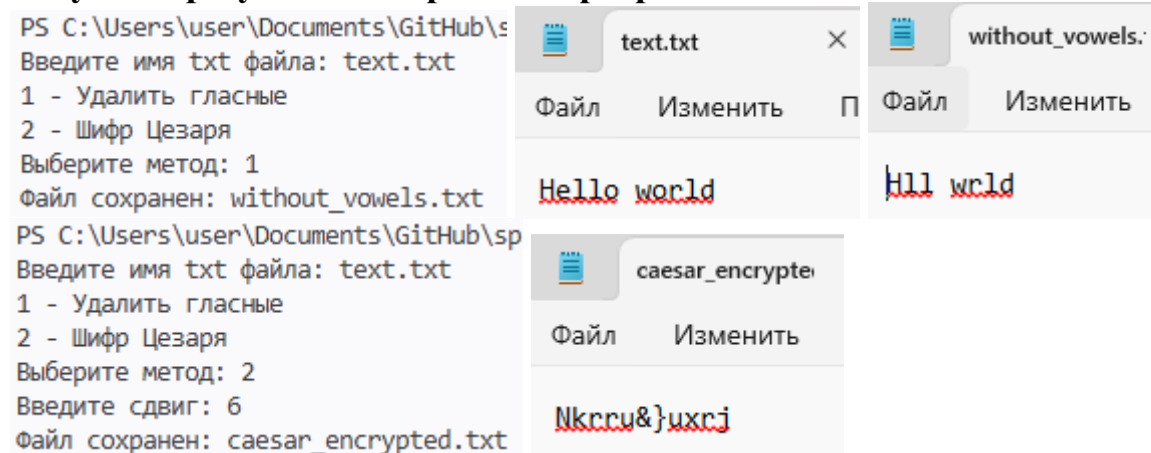
    with open("caesar_encrypted.txt", "w", encoding="utf-8") as file:
        file.write(result)

    print("Файл сохранен: caesar_encrypted.txt")

else:
    print("Неверный выбор")

```

Рисунки с результатами работы программы



Вывод: приобрёл навыки применения паттернов проектирования при решении практических задач с использованием языка Python